

Functional Verification of Benford_py Five Years After Its Release

Harsh Torane

13 September 2026

Abstract

Benford’s Law describes the expected frequency distribution of leading digits in many naturally occurring datasets. The open-source Python package *benford_py* implements statistical tests for conformity with this principle. Despite its final release (v0.5.0) in June 2021 and minimal subsequent maintenance, the package’s documented workflow remains functional on a contemporary Python stack. This study verifies the workflow using a fresh repository clone and modern, unpinned dependencies. The core analysis produces expected first-digit distributions; however, the test suite fails due to a NumPy 2.0 incompatibility, and an undocumented warning appears during plotting. These findings demonstrate the package’s continued analytical utility while highlighting critical maintenance needs in its testing infrastructure. This case study examines one specific GitHub repository five years after its final release; the findings are particular to *benford_py* and cannot be generalized to unmaintained scientific packages within the broader ecosystem.

1. Introduction

Benford’s Law, formally known as the First-Digit Law, describes the expected frequency distribution of leading digits in many naturally occurring datasets, with smaller digits appearing more frequently than larger ones. This principle has proven valuable in finance, accounting, forensics, and fraud detection applications.

The *benford_py* package provides statistical tests for assessing conformity with Benford’s Law. Notably, the package lacks an associated peer-reviewed publication; its `CITATION.cff` file references only the GitHub repository. The final tagged release (v0.5.0) was published in June 2021, and the repository’s master branch has since received only a single commit—a license-header correction in October 2022 that introduced no functional changes.

This study evaluates whether *benford_py* continues to execute its documented workflow correctly within a modern Python environment, addressing broader concerns about software sustainability and decay in unmaintained scientific packages.

2. Software Under Test

Property	Value
Repository	github.com/milcent/benford_py
Commit	0126c606ae9c27cba43e6dc83b73bb329f839ae4
Commit date	11 October 2022
Final tagged release	v0.5.0 (June 2021)
Python interpreter	3.13.5
Dependencies	pandas 3.0.5, NumPy 2.5.3, Matplotlib 3.11.2
Sample dataset	<code>data/SPY.csv</code> (bundled)
Test suite status	Collection failure: <code>np.float_</code> reference (removed in NumPy 2.0)
Verification timestamp	13 September 2026, 19:46 IST

3. Methodology

To assess *benford_py*'s current functionality, the following steps were undertaken:

1. **Environment Setup:** A fresh clone of the master branch was obtained, and a disposable virtual environment was created. The three primary dependencies—pandas, NumPy, and Matplotlib—were installed without version pinning to simulate a contemporary Python stack.
2. **Workflow Execution:** The quick-start workflow, as documented in the project's README and demonstration notebook, was executed using the bundled `data/SPY.csv` dataset. This involved:
 - Loading the dataset
 - Computing simple and logarithmic returns
 - Running a first-digit Benford test
3. **Test Suite Evaluation:** The package's pytest suite was executed to determine whether it remains functional.
4. **Evidence Capture:** The resolved commit metadata, the complete dependency environment (via `pip freeze`), and the raw workflow and pytest output were captured for independent verification and reproducibility (detailed in Appendix A).

4. Results

4.1 Workflow Execution

The documented workflow executed successfully without errors:

- The sample data loaded correctly.
- Simple and logarithmic returns were computed as expected.
- The first-digit Benford test processed **5,968 registries** with **0 records discarded**.

First-digit distribution:

Digit	Count	Found (%)	Expected (%)
1	1835	30.75	30.10
2	949	15.90	17.61
3	668	11.19	12.49
4	534	8.95	9.69
5	494	8.28	7.92
6	447	7.49	6.69
7	386	6.47	5.80
8	345	5.78	5.12
9	310	5.19	4.58

The observed distribution closely matches theoretical expectations. Additionally, an undocumented `UserWarning` was raised during the plotting phase, indicating a discrepancy between the documented behavior and the actual runtime implementation.

4.2 Test Suite Evaluation

The pytest suite failed during the collection phase due to a reference to `np.float_` in `tests/conftest.py:174`, which raises an `AttributeError` in NumPy 2.0 (`np.float_` was removed; the recommended replacement is `np.float64`). This issue does not affect runtime functionality or the accuracy of the workflow results but prevents test execution entirely.

5. Discussion

The successful execution of *benford_py*'s documented workflow demonstrates that its core functionality remains compatible with modern Python environments, despite four years without functional updates. The observed first-digit distribution aligns closely with Benford's Law expectations, confirming the package's analytical validity.

However, two issues warrant attention:

1. **Test Suite Failure:** The deprecated `np.float_` reference prevents test execution, revealing a significant maintenance gap in the testing infrastructure. While this does not directly impact runtime behavior or analytical results, it undermines confidence in the package's long-term reliability and makes it impossible to verify that future dependency updates will not introduce breaking changes.
2. **Undocumented Behavior:** The `UserWarning` during plotting suggests undocumented behaviors that could mislead users who rely solely on the official documentation. Such discrepancies should be formally documented or resolved to ensure transparency and consistency.

These findings underscore the importance of regular maintenance in scientific software, even when core functionality appears stable and functional. The study also demonstrates that workflow functionality does not guarantee test suite compatibility, emphasizing the critical need for comprehensive verification across all package components.

6. Conclusion

Five years after its final release, *benford_py* presents a nuanced picture of software aging: its core runtime execution remains remarkably robust on a modern Python stack, yet its development infrastructure has decayed. While practitioners can still successfully execute quick first-digit conformity checks using unpinned modern dependencies, unresolved blockers—such as the NumPy 2.0 test suite incompatibility—prevent systematic verification. Ultimately, while this single-repository audit cannot be generalized to the broader scientific Python ecosystem, it illustrates how analytical utility can persist long past active development, provided maintainers eventually step in to modernize underlying testing frameworks.

7. Data and Code Availability

No novel data were generated for this study. The software under test, *benford_py*, is publicly available at github.com/milcent/benford_py (commit 0126c606ae9c27cba43e6dc83b73bb329f839ae4). The sample dataset (`data/SPY.csv`) is included in the repository. Complete reproduction commands are provided in Appendix B.

8. Conflicts of Interest

The author declares no conflicts of interest.

9. Funding

This study was self-funded by the author.

Appendix A: Verification Evidence

Commit Provenance

The tested commit was inspected directly via `git log` to substantiate the claim that no functional change has occurred since the v0.5.0 release:

```
commit 0126c606ae9c27cba43e6dc83b73bb329f839ae4
Merge: 8afb27 23893ce
Author: Marcel Milcent <marcelmilcent@gmail.com>
AuthorDate: Tue Oct 11 05:35:46 2022 -0300
Commit: GitHub <noreply@github.com>
CommitDate: Tue Oct 11 05:35:46 2022 -0300
```

```
Merge pull request #56 from hyandell/patch-1
Relicensing per earlier LICENSE change
```

Computational Environment

Verification was performed on 13 September 2026 at 19:46 IST (2026-09-13T19:46:23+05:30) using Python 3.13.5 in a disposable virtual environment. The complete resolved dependency set was:

```
contourpy==1.4.0
cycler==0.12.1
fonttools==4.65.0
iniconfig==2.3.0
kiwisolver==1.5.1
matplotlib==3.11.2
numpy==2.5.3
packaging==26.3
pandas==3.0.5
pillow==12.3.0
pluggy==1.6.0
Pygments==2.21.0
pyparsing==3.3.2
pytest==9.1.1
python-dateutil==2.9.0.post0
six==1.17.0
```

Raw Workflow Output

The quick-start workflow produced the following output verbatim, including the undocumented `UserWarning`:

```
/tmp/benford_verification_20260913_194157/benford_py/benford/viz.py:134:
UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show(block=False)
```

Initialized sequence with 5968 registries.

Test performed on 5968 registries.

Discarded 0 records < 1 after preparation.

	Counts	Found	Expected
First_1_Dig			
1	1835	0.307473	0.301030
2	949	0.159015	0.176091
3	668	0.111930	0.124939

4	534	0.089477	0.096910
5	494	0.082775	0.079181
6	447	0.074899	0.066947
7	386	0.064678	0.057992
8	345	0.057808	0.051153
9	310	0.051944	0.045757

Raw Pytest Output

Execution of the package's test suite (`pytest tests/`) failed during the collection phase:

ImportError while loading conftest

'/tmp/benford_verification_20260913_194157/benford_py/tests/conftest.py'.

tests/conftest.py:174: in <module>

```
(gen_mantissa_distribution(), np.random.choice([True, False]), np.float_)
~~~~~~
```

benford_env/lib/python3.13/site-packages/numpy/__init__.py:763: in __getattr__

```
raise AttributeError(
```

E AttributeError: `np.float\_` was removed in the NumPy 2.0 release.

Use `np.float64` instead.

This is a collection-time `AttributeError` raised by a deprecated NumPy alias referenced in the test fixtures, not a failure arising from any assertion in the package's runtime source code.

Appendix B: Reproducibility Commands

The following commands reproduce the verification described in this study:

```
# 1. Obtain the exact commit under test
git clone https://github.com/milcent/benford_py.git
cd benford_py
git checkout 0126c606ae9c27cba43e6dc83b73bb329f839ae4

# 2. Build a disposable, unpinned environment
python3 -m venv benford_env
./benford_env/bin/pip install --upgrade pip
./benford_env/bin/pip install pandas numpy matplotlib pytest

# 3. Primary check -- documented quick-start workflow
./benford_env/bin/python - <<'PY'
import matplotlib
matplotlib.use('Agg')
import numpy as np
import pandas as pd
import benford as bf

sp = pd.read_csv(
    'data/SPY.csv',
    index_col='Date',
    parse_dates=True
)
sp.rename(columns={'Adj Close': 'Adj_Close'}, inplace=True)
sp['p_r'] = sp.Close / sp.Close.shift() - 1
sp['l_r'] = np.log(sp.Close / sp.Close.shift())

f1d = bf.first_digits(sp.l_r, digs=1, decimals=8)
print(f1d)
PY

# 4. Secondary check -- test suite
./benford_env/bin/python -m pytest tests/

# The experiment intentionally uses unpinned dependencies to test the package
# against a contemporary Python stack. The exact environment used for the
# reported results is recorded in Appendix A.
```

Step 3 reproduces the first-digit distribution and `UserWarning` reported in Section 4.1. Step 4 reproduces the `np.float_` collection failure reported in Section 4.2. For exact reproducibility, explicitly pin versions: `pip install pandas==3.0.5 numpy==2.5.3 matplotlib==3.11.2 pytest==9.1.1`.

Verification date: 13 September 2026.